Лабораторная работа №2

Дерево объектов Qt и автоматическое удаление виджетов

Цель работы: Изучить механизм дерева объектов Qt, принцип владения объектами через родителя, автоматическое удаление дочерних объектов при уничтожении родителя, а также разобраться в корректном и некорректном использовании setParent() в приложениях Qt Widgets.

Задачи:

1. Изучить понятие дерева объектов QObject.
2. Понять, как родитель владеет дочерними объектами.
3. Научиться создавать виджеты с явным родителем.
4. Реализовать окно минимум с 5 виджетами.
5. Отследить создание и уничтожение объектов через qDebug() в деструкторах.
6. Продемонстрировать автоматическое удаление дочерних виджетов при удалении родителя.
7. Исследовать поведение setParent() при смене владельца виджета.
8. Разобрать типичные ошибки, связанные с setParent(), удалением объектов и висячими указателями.
9. Оформить отчёт и загрузить проект в Git-репозиторий.

Теоретическое обоснование

1. Дерево объектов QObject

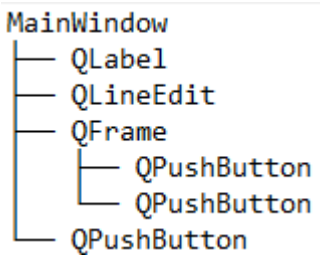
Практически все классы Qt наследуются от QObject. Каждый QObject может иметь:

– одного родителя;

– произвольное количество детей.

Это образует дерево объектов.

Пример:



Родитель владеет своими детьми. Это означает, что при уничтожении родителя автоматически уничтожаются все его дочерние объекты.

2. Автоматическое управление памятью в Qt

В Qt часто используется следующая схема:

QPushButton *button = new QPushButton("OK", parentWidget);

Здесь button создаётся в куче, но имеет родителя parentWidget.

После этого вручную удалять button обычно не нужно. Когда будет удалён parentWidget, Qt автоматически удалит всех своих детей, включая button.

То есть работает принцип:

Владелец объекта – его родитель.

3. QWidget и визуальное дерево

QWidget наследует не только свойства QObject, но и добавляет визуальную иерархию.

Родитель виджета одновременно является:

– владельцем объекта;

– визуальным контейнером;

– участником системы координат;

– элементом цепочки обработки событий.

Например:

```
QWidget window;
QLabel label("Hello", &window);
```

label будет отображаться внутри window, а при уничтожении window метка label будет автоматически удалена.

4. Метод setParent()

Функция setParent() позволяет изменить родителя объекта.

Для виджетов это означает сразу несколько вещей:

1. Меняется владелец объекта.

2. Меняется визуальный родитель.

3. Виджет может быть удалён из старого layout.

4. Виджет может потребовать повторного добавления в новый layout.

5. Виджет может стать независимым окном, если передать nullptr.

Пример:

```
widget->setParent(newParent);
```

После этого newParent становится владельцем widget.

Если же вызвать:

```
widget->setParent(nullptr);
```

то объект отвязывается от прежнего владельца и становится независимым. В этом случае кто-то должен явно удалить его через delete или deleteLater().

5. delete и deleteLater

В Qt есть два основных способа удаления объекта:

```
delete object;
```

и

```
object->deleteLater();
```

Разница важна.

delete

Удаляет объект немедленно.

Опасно использовать, если объект участвует в текущем вызове стека, например:

– находится в обработчике сигнала;

– является отправителем сигнала;

– может быть удалён вместе с родителем;

– на него ещё могут ссылаться другие объекты.

deleteLater()

Ставит объект в очередь на удаление. Реальное уничтожение произойдёт, когда управление вернётся в цикл событий Qt.

Это безопаснее при работе с сигналами, слотами и динамическим созданием/удалением виджетов.

Пример:

panel->deleteLater();

6. Отслеживание деструкторов через qDebug()

Чтобы убедиться, что объекты действительно удаляются, можно создать производные классы виджетов и переопределить их деструкторы.

Пример:

```
class LogLabel : public QLabel
{
public:
    explicit LogLabel(const QString &text, QWidget *parent = nullptr)
        : QLabel(text, parent)
    {
        qDebug() << "[CREATE] LogLabel" << text;
    }

    ~LogLabel() override
    {
        qDebug() << "[DESTROY] LogLabel" << text();
    }
};
```

При закрытии окна или удалении родителя в консоли появятся сообщения о вызове деструкторов.

Важно понимать порядок уничтожения.

В C++ сначала выполняется тело деструктора производного класса, затем разрушаются члены класса, затем вызываются деструкторы базовых

классов. Удаление детей QObject происходит в деструкторе QObject, то есть базового класса.

Поэтому сообщение:

[DESTROY] Parent

может появиться раньше, чем сообщения об удалении детей. Это нормально. Главное – дети действительно уничтожаются вместе с родителем.

Пример выполнения с пошаговым описанием

Шаг 1. Создание проекта

В Qt Creator:

File → New File or Project → Application → Qt Widgets Application

Имя проекта:

ObjectTreeDemo

Система сборки:

qmake

Базовый класс главного окна можно выбрать QWidget, так как нам не обязательно использовать QMainWindow.

Шаг 2. Настройка .pro-файла

Файл ObjectTreeDemo.pro:

```
QT += widgets

CONFIG += c++17
CONFIG -= app_bundle

TARGET = ObjectTreeDemo
TEMPLATE = app

SOURCES += \
    main.cpp \
    mainwindow.cpp

HEADERS += \
```

```
    mainwindow.h \
    widgets.h

FORMS += \
    widgets.ui

# Default rules for deployment.
qnx: target.path = /tmp/$${TARGET}/bin
else: unix:!android: target.path = /opt/$${TARGET}/bin
!isEmpty(target.path): INSTALLS += target
```

Комментарии:

– QT += widgets – подключаем GUI-модуль.

– CONFIG += console – полезно в Windows, чтобы видеть вывод qDebug() в консоли.

– CONFIG -= app_bundle – актуально для macOS.

– В SOURCES перечисляем только .cpp-файлы.

– В HEADERS перечисляем заголовочные файлы.

Шаг 3. Создание отслеживаемых виджетов

Создадим файл widgets.h.

В нём будут простые производные классы, которые печатают сообщения при создании и уничтожении.

```
#ifndef WIDGETS_H
#define WIDGETS_H

#include <QDebug>
#include <QFrame>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QWidget>

class LogLabel : public QLabel
{
public:
    explicit LogLabel(const QString &text, QWidget *parent = nullptr)
        : QLabel(text, parent)
```

```
    {
        qDebug() << "[CREATE] LogLabel" << text;
    }

    ~LogLabel() override
    {
        qDebug() << "[DESTROY] LogLabel" << text();
    }
};

class LogButton : public QPushButton
{
public:
    explicit LogButton(const QString &text, QWidget *parent = nullptr)
        : QPushButton(text, parent)
    {
        qDebug() << "[CREATE] LogButton" << text;
    }

    ~LogButton() override
    {
        qDebug() << "[DESTROY] LogButton" << text();
    }
};

class LogLineEdit : public QLineEdit
{
public:
    explicit LogLineEdit(QWidget *parent = nullptr)
        : QLineEdit(parent)
    {
        qDebug() << "[CREATE] LogLineEdit";
    }

    ~LogLineEdit() override
    {
        qDebug() << "[DESTROY] LogLineEdit";
    }
};

class LogPanel : public QFrame
{
public:
    explicit LogPanel(const QString &title, QWidget *parent = nullptr)
        : QFrame(parent)
    {
        setObjectName(title);
        setFrameShape(QFrame::StyledPanel);

        qDebug() << "[CREATE] LogPanel" << title;
    }
```

```
    ~LogPanel() override
    {
        qDebug() << "[DESTROY] LogPanel" << objectName();
    }
};

#endif // WIDGETS_H
```

Комментарии:

– Мы не создаём отдельные .cpp-файлы для этих классов, так как реализация маленькая.

– Каждый деструктор выводит сообщение через qDebug().

– LogPanel использует objectName() для удобной отладки.

– Все классы принимают parent в конструкторе.

Шаг 4. Заголовок главного окна

Файл mainwindow.h:

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QPointer>
#include <QVBoxLayout>
#include <QWidget>

#include "widgets.h"

class MainWindow : public QWidget
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow() override;

private slots:
    void moveMovableWidget();
    void deletePanel();
    void demoDetaching();

private:
    void setupUi();
    void setStatus(const QString &text);

    QPointer<QVBoxLayout> m_rootLayout;
    QPointer<QVBoxLayout> m_panelLayout;
```

```
    QPointer<LogLabel> m_title;
    QPointer<LogLineEdit> m_input;
    QPointer<LogLabel> m_status;

    QPointer<LogPanel> m_panel;
    QPointer<LogButton> m_movable;

    QPointer<LogButton> m_moveButton;
    QPointer<LogButton> m_deleteButton;
};

#endif // MAINWINDOW_H
```

Комментарии:

– Q_OBJECT нужен, потому что в классе есть слоты.

– QPointer используется, чтобы безопасно проверять, жив ли объект.

– m_movable – виджет, который мы будем перемещать между родителем и панелью.

– m_panel – панель-владелец, которую мы будем удалять.

Шаг 5. Реализация главного окна

Файл mainwindow.cpp:

```
#include "mainwindow.h"

#include <QDebug>

MainWindow::MainWindow(QWidget *parent)
    : QWidget(parent)
{
    setupUi();
}

MainWindow::~MainWindow()
{
    qDebug() << "[DESTROY] MainWindow";
}

void MainWindow::setupUi()
{
    setObjectName("MainWindow");
    setWindowTitle("Qt Object Tree Demo");
    resize(520, 640);
```

```
            m_rootLayout = new QVBoxLayout(this);

            m_title = new LogLabel("Дерево объектов Qt: владелец и потомки",
this);
            m_rootLayout->addWidget(m_title.data());

            m_input = new LogLineEdit(this);
            m_input->setPlaceholderText("Виджет-ребёнок главного окна");
            m_rootLayout->addWidget(m_input.data());

            m_status = new LogLabel("Статус: окно создано", this);
            m_status->setWordWrap(true);
            m_rootLayout->addWidget(m_status.data());

            m_panel = new LogPanel("Панель-владелец", this);
            m_rootLayout->addWidget(m_panel.data());

            m_panelLayout = new QVBoxLayout(m_panel.data());

            LogButton *staticButton =
                new LogButton("Статическая кнопка внутри панели",
m_panel.data());
            m_panelLayout->addWidget(staticButton);

            m_movable = new LogButton("Перемещаемый виджет (в панели)",
m_panel.data());
            m_panelLayout->addWidget(m_movable.data());

            m_moveButton = new LogButton("Переместить виджет: панель <-> окно",
this);
            connect(m_moveButton.data(), &QPushButton::clicked,
                    this, &MainWindow::moveMovableWidget);
            m_rootLayout->addWidget(m_moveButton.data());

            m_deleteButton = new LogButton("Удалить панель (deleteLater)",
this);
            connect(m_deleteButton.data(), &QPushButton::clicked,
                    this, &MainWindow::deletePanel);
            m_rootLayout->addWidget(m_deleteButton.data());

            LogButton *detachButton = new LogButton("Демо setParent(nullptr)",
this);
            connect(detachButton, &QPushButton::clicked,
                    this, &MainWindow::demoDetaching);
            m_rootLayout->addWidget(detachButton);

            m_rootLayout->addStretch();
```

```
        setStatus(
            "Готово. Все виджеты созданы с родителями. "
            "Удаление родителя автоматически удалит детей."
        );
    }

    void MainWindow::setStatus(const QString &text)
    {
        if (!m_status.isNull()) {
            m_status->setText(text);
        }
    }

    void MainWindow::moveMovableWidget()
    {
        if (m_movable.isNull()) {
            setStatus("Перемещаемый виджет уже уничтожен.");
            return;
        }

        if (m_panel.isNull()) {
            setStatus("Панель уже удалена. Вернуть виджет в неё
невозможно.");
            return;
        }

        if (m_movable->parentWidget() == m_panel.data()) {
            if (!m_panelLayout.isNull()) {
                m_panelLayout->removeWidget(m_movable.data());
            }

            m_movable->setParent(this);

            if (!m_rootLayout.isNull()) {
                m_rootLayout->addWidget(m_movable.data());
            }

            m_movable->setText("Перемещаемый виджет (вне панели)");

            setStatus(
                "setParent(this) сделал главным окном новым владельцем. "
                "Панель больше не удалит этот виджет автоматически."
            );
        } else {
            if (!m_rootLayout.isNull()) {
                m_rootLayout->removeWidget(m_movable.data());
            }
```

```
            m_movable->setParent(m_panel.data());

            if (!m_panelLayout.isNull()) {
                m_panelLayout->addWidget(m_movable.data());
            }

            m_movable->setText("Перемещаемый виджет (в панели)");

            setStatus(
                "Виджет снова стал ребёнком панели. "
                "Теперь он будет удалён вместе с панелью."
            );
        }

        m_movable->show();

        qDebug() << "[SET_PARENT] Новый владелец виджета:"
                 << m_movable->parentWidget()->objectName();
    }

    void MainWindow::deletePanel()
    {
        if (m_panel.isNull()) {
            setStatus("Панель уже удалена.");
            return;
        }

        if (!m_movable.isNull() && m_movable->parentWidget() ==
m_panel.data()) {
            setStatus(
                "Панель будет удалена. Перемещаемый виджет является её
ребёнком, "
                "поэтому будет уничтожен автоматически."
            );
        } else {
            setStatus(
                "Панель будет удалена. Перемещаемый виджет не является её
ребёнком, "
                "поэтому останется жив."
            );
        }

        if (!m_deleteButton.isNull()) {
            m_deleteButton->setEnabled(false);
        }

        if (!m_moveButton.isNull()) {
            m_moveButton->setEnabled(false);
```

```
        }

        qDebug() << "[DELETE_LATER] Запрошено удаление панели:"
                 << m_panel->objectName();

        m_panel->deleteLater();
    }

    void MainWindow::demoDetaching()
    {
        LogPanel *tempOwner = new LogPanel("TempOwner", this);
        tempOwner->hide();

        new LogLabel("Ребёнок TempOwner", tempOwner);

        LogLabel *detachedChild =
            new LogLabel("Отвязанный ребёнок TempOwner", tempOwner);

        qDebug() << "[DEMO] Количество детей tempOwner до detach:"
                 << tempOwner->children().count();

        detachedChild->setParent(nullptr);

        qDebug() << "[DEMO] Количество детей tempOwner после detach:"
                 << tempOwner->children().count();

        setStatus(
            "Демо: одна метка осталась ребёнком TempOwner, "
            "другая отвязана через setParent(nullptr). "
            "При удалении TempOwner автоматическое удаление применится
только к первой."
        );

        tempOwner->deleteLater();
        detachedChild->deleteLater();
    }
```

Шаг 6. Файл main.cpp

```
    #include <QApplication>

    #include "mainwindow.h"

    int main(int argc, char *argv[])
    {
        QApplication app(argc, argv);

        MainWindow window;
```

```
    window.show();

    return app.exec();
}
```

Комментарии:

– Для GUI-приложения используется QApplication.

– app.exec() запускает цикл событий Qt.

– Без цикла событий не будут работать, например, deleteLater(), таймеры, обработка кликов и перерисовка окна

Шаг 7. Запуск и проверка

Запустите приложение.

В консоли или в окне Application Output в Qt Creator вы увидите сообщения вида:

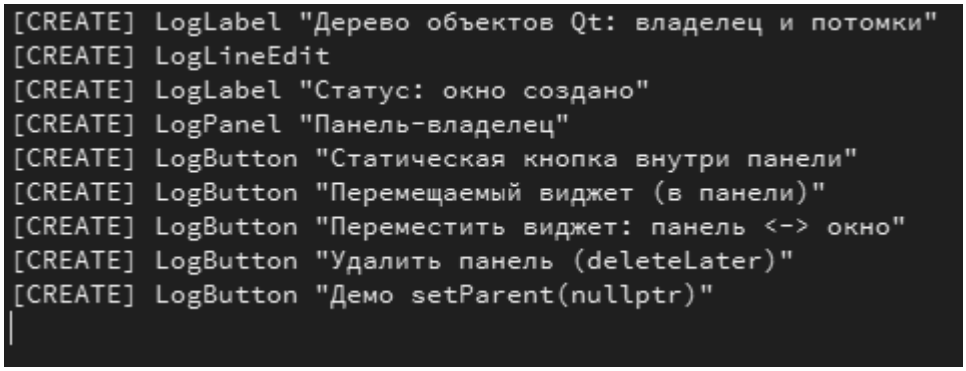


Рисунок 1. Консоль

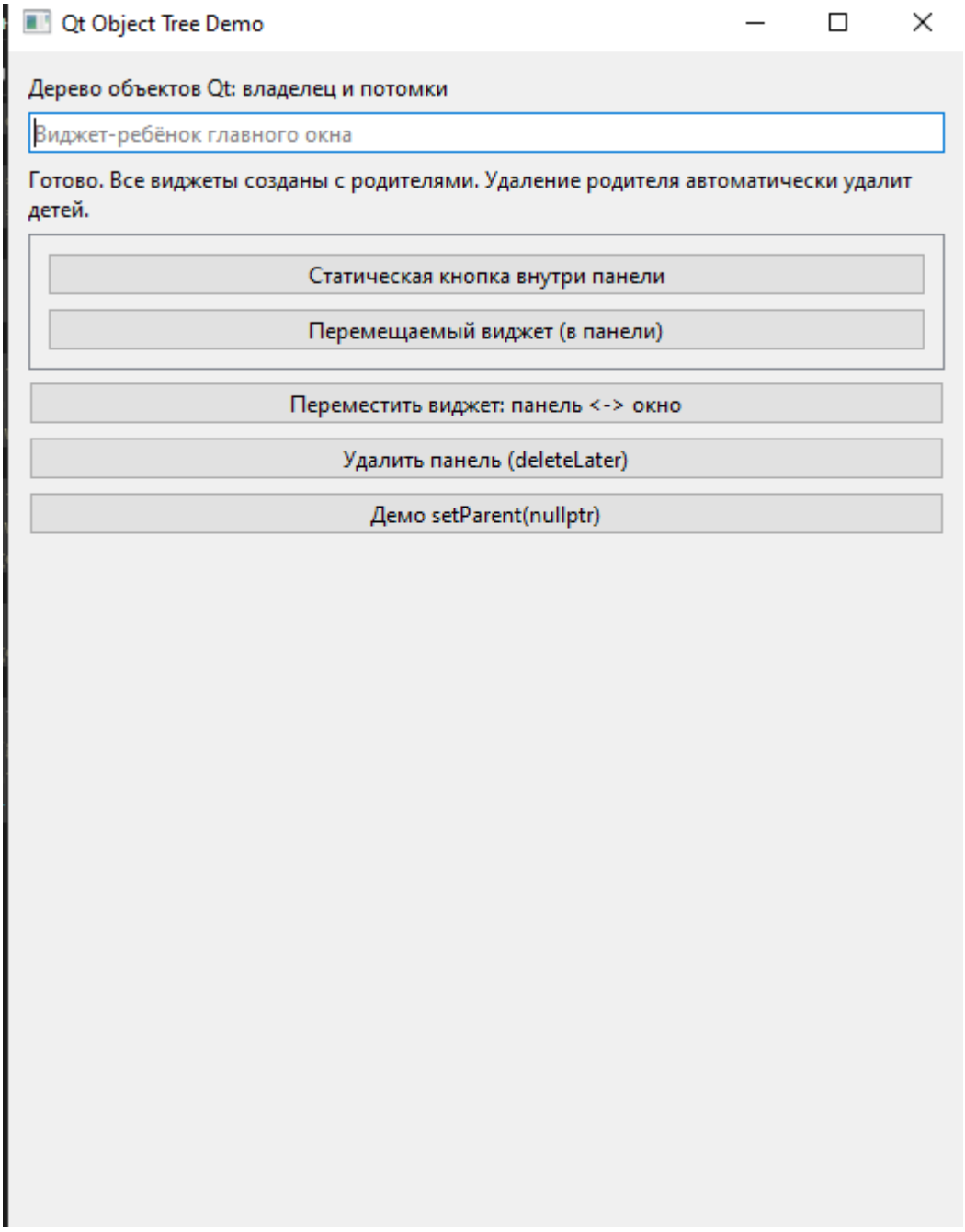


Рисунок 2. Созданное окно

Это подтверждает, что все виджеты созданы.

Шаг 8. Проверка автоматического удаления Вариант А. Удалить панель, не перемещая виджет

1. Запустите программу.

2. Не нажимайте кнопку перемещения.

3. Нажмите:

Удалить панель (deleteLater)

В консоли появятся сообщения примерно такого вида:

```
[DELETE_LATER] Запрошено удаление панели: "Панель-владелец"
[DESTROY] LogPanel "Панель-владелец"
[DESTROY] LogButton "Статическая кнопка внутри панели"
[DESTROY] LogButton "Перемещаемый виджет (в панели)"
```

Рисунок 3. Консоль

Порядок удаления детей может отличаться. Это нормально.

Главный вывод: мы не удаляли кнопки вручную. Они были уничтожены автоматически, потому что их родителем была панель.

Вариант B. Сначала переместить виджет, потом удалить панель

1. Запустите программу.

2. Нажмите:

Переместить виджет: панель <-> окно

3. Затем нажмите:

Удалить панель (deleteLater)

Теперь перемещаемый виджет не будет удалён вместе с панелью, потому что он сменил родителя.

В консоли можно увидеть:

[SET_PARENT] Новый владелец виджета: "MainWindow"

[DELETE_LATER] Запрошено удаление панели: "Панель-владелец"

[DESTROY] LogPanel "Панель-владелец"

[DESTROY] LogButton "Статическая кнопка внутри панели"

Перемещаемый виджет останется жив, потому что его родителем стало главное окно.

Вариант C. Демо setParent(nullptr)

Нажмите кнопку:

Демо setParent(nullptr)

В консоли появится что-то вроде:

[CREATE] LogPanel "TempOwner"

[CREATE] LogLabel "Ребёнок TempOwner"

[CREATE] LogLabel "Отвязанный ребёнок TempOwner"

[DEMO] Количество детей tempOwner до detach: 2

[DEMO] Количество детей tempOwner после detach: 1

[DESTROY] LogPanel "TempOwner"

[DESTROY] LogLabel "Ребёнок TempOwner"

[DESTROY] LogLabel "Отвязанный ребёнок TempOwner"

Здесь важно:

– первый LogLabel остался ребёнком TempOwner;

– второй LogLabel был отвязан через setParent(nullptr);

– после удаления TempOwner первый ребёнок уничтожился автоматически;

– второй ребёнок требовал отдельного удаления, поэтому в примере вызван deleteLater() для него.

Разбор ошибок setParent()

В этой части лабораторной необходимо продемонстрировать и объяснить минимум 3 типичные ошибки.

Ошибка 1. Стековый объект как ребёнок кучного родителя

Некорректный пример:

```
void badExample()
{
    QWidget *parent = new QWidget;

    QLabel child("Danger", parent);

    delete parent;
}
```

Что происходит:

1. parent создан в куче.

2. child создан на стеке, но имеет родителя parent.

3. При delete parent Qt пытается удалить всех детей родителя.

4. Среди детей оказывается стековый объект child.

5. Qt вызывает деструктор для объекта, который уже будет уничтожен при выходе из области видимости.

6. Возможно двойное уничтожение и падение программы.

Правильно:

```
void goodExample1()
{
    QWidget parent;

    QLabel *child = new QLabel("Safe", &parent);

    // Удалять child вручную не нужно.
    // При уничтожении parent child будет удалён автоматически.
}
```

Или:

```
void goodExample2()
{
    QWidget *parent = new QWidget;

    QLabel *child = new QLabel("Safe", parent);

    delete parent;
}
```

Правило:

Не делайте стековый объект ребёнком кучного родителя.

Ошибка 2. Потеря владельца после setParent(nullptr)

Некорректный пример:

```
QWidget *widget = new QWidget;
widget->setParent(nullptr);

// Забыли удалить widget
```

Что происходит:

– объект был отвязан от родителя;

– никто больше не владеет им;

– возникает утечка памяти.

Правильно:

```
QWidget *widget = new QWidget;
widget->setParent(nullptr);

widget->deleteLater();
```

Или просто не отвязывать объект, если за него должен отвечать родитель.

Правило:

После setParent(nullptr) объект становится независимым. Его нужно удалить вручную или через deleteLater().

Ошибка 3. Использование указателя на ребёнка после удаления родителя

Некорректный пример:

```
LogPanel *panel = new LogPanel("Panel", this);
LogButton *button = new LogButton("Child", panel);

panel->deleteLater();

button->setText("Crash?");
```

Если button была ребёнком panel, то после удаления панели указатель button станет висячим.

Правильно использовать QPointer:

```
QPointer<LogButton> button = new LogButton("Child", panel);

panel->deleteLater();

if (!button.isNull()) {
    button->setText("Safe");
}
```

Правило:

Если объект может быть удалён вместе с родителем, храните указатель через QPointer или обнуляйте его при сигнале destroyed().

Ошибка 4. Неправильное перемещение виджета между layout

Некорректно просто вызвать:

widget->setParent(newParent);

и ожидать, что виджет автоматически появится в новом layout.

Правильный порядок:

```
oldLayout->removeWidget(widget);

widget->setParent(newParent);

newLayout->addWidget(widget);

widget->show();
```

Именно так сделано в методе moveMovableWidget().

Правило:

При смене родителя виджета нужно явно обновлять layout.

Ошибка 5. Немедленное delete объекта внутри обработчика сигнала

Опасно:

```
connect(button, &QPushButton::clicked,
        panel, &QWidget::delete);
```

Если button находится внутри panel, то при нажатии кнопки может произойти удаление объекта, который сейчас участвует в обработке сигнала.

Правильно:

```
connect(button, &QPushButton::clicked,
        panel, &QWidget::deleteLater);
```

Правило:

Внутри сигналов и слотов предпочтительно использовать deleteLater(), а не delete.

Ошибка 6. setParent между разными потоками

Дерево объектов Qt привязано к потокам. Нельзя безопасно делать объект одного потока ребёнком объекта другого потока.

Признак проблемы:

QObject::setParent: Cannot set parent, new parent is in a different thread

Правильно:

– объекты одного дерева должны находиться в одном потоке;

– для межпоточного взаимодействия используйте сигналы и слоты;

– при необходимости используйте moveToThread() до установки родителя.

Контрольные вопросы

1. Что такое дерево объектов QObject?

2. Кто является владельцем дочернего объекта?

3. Что произойдёт при удалении родителя?

4. Чем QWidget-дерево отличается от простого QObject-дерева?

5. Зачем передавать parent в конструктор виджета?

6. Что делает setParent()?

7. Что произойдёт при вызове setParent(nullptr)?

8. Можно ли вручную удалять объект, у которого есть родитель?

9. Почему опасно удалять объект через delete внутри обработчика его собственного сигнала?

10. Чем deleteLater() отличается от delete?

11. Что такое висячий указатель?

12. Как QPointer помогает избежать работы с удалённым объектом?

13. Почему стековый объект нельзя делать ребёнком кучного родителя?

14. Как правильно перемещать виджет из одного layout в другой?

15. Что произойдёт, если удалить родителя, а затем обратиться к указателю на ребёнка?

16. Как отследить момент уничтожения объекта через деструктор?

17. Почему порядок вызова деструкторов родителя и детей может быть неочевидным?

18. Что возвращает children() у QObject?

19. Чем findChildren<QWidget*>() отличается от children()?

20. Можно ли иметь у объекта несколько родителей?

21. Как связано дерево объектов и цикл событий Qt?

22. Почему для GUI-приложения нужен QApplication и app.exec()?

23. Что произойдёт, если попытаться сделать родителем объект из другого потока?

24. Как проверить отсутствие утечек памяти?

25. Почему в данной работе используется QPointer, а не обычные указатели?

Требования к отчёту

Отчёт должен быть оформлен в формате PDF (или Markdown с конвертацией в PDF) и содержать следующие разделы:

1. Титульный лист: название дисциплины, тема лабораторной работы, ФИО студента, группа, ФИО преподавателя, дата.

2. Цель работы (из методических указаний).

3. Постановка задачи.

4. Теоретическое обоснование: краткое описание метаобъектной системы Qt (не более 2 страниц).

5. Листинг кода с комментариями.

6. Пример работы программы: скриншоты или текст сессии консоли с 5-10 примерами команд.

7. Разбор ошибок и их решений: опишите 2-3 проблемы, возникшие в ходе работы, и как вы их решили.

8. Вывод: что нового изучили, какие сложности возникли, что осталось непонятным.

9. Ссылка на Git-репозиторий – обязательное требование.

Требование к репозиторию

Проект должен быть загружен в публичный Git-репозиторий (GitHub, GitLab, Bitbucket, или собственный сервер).

Требования к репозиторию:

1. Наличие файла README.md с описанием проекта, инструкцией по

сборке и запуску.

2. Наличие файла .gitignore (исключить папки build/, debug/, release/, *.user, *.o, moc_*, *.exe).

3. Осмысленные сообщения коммитов (не «fix», «update», «test», а «Добавлен слот divide с проверкой деления на ноль»).

4. Минимум 3-5 коммитов, отражающих прогресс работы.

5. Все исходные файлы (.pro, .h, .cpp) и НЕ файлы сборки.

6. Опционально: тесты, документация Doxygen.

7. Ссылку на репозиторий вставьте в раздел «Требования к отчёту» или на титульный лист.